

Lab 2. Numpy

Task 1. (random matrix and the main diagonal)

- Generate a random 10×10 matrix of integers between 0 and 99.
- Replace every value on the main diagonal with the maximum value of its column.

A function `np.fill_diagonal` may be useful.

Perform all computations without loops!

```
In [1]: import numpy as np
matrix=np.random.randint(0,99,(10,10))
print(f"Before replacing:\n{matrix}")
np.fill_diagonal(matrix, np.max(matrix, axis=0))
print(f"\nAfter replacing:\n{matrix}")
```

Before replacing:

```
[[ 9 44 64 69 19 87 10 36 65 88]
 [32  4 42 30 89 81 38 39 64 94]
 [81 46 26  0 79 31 52 76 91 80]
 [44  5 32 17 37 10 71 95 82 76]
 [25 46 65 87 11 18 53 78 14 35]
 [39 92 42 59 96 54 41 80 97 92]
 [54 62 72 43 62 38 78 63 31 58]
 [31 52 82 16 83 96 94 90 83 49]
 [13 21 10 75 65 20 45  5 82  0]
 [74 39 86 55 88 17 61 76 49 97]]
```

After replacing:

```
[[81 44 64 69 19 87 10 36 65 88]
 [32 92 42 30 89 81 38 39 64 94]
 [81 46 86  0 79 31 52 76 91 80]
 [44  5 32 87 37 10 71 95 82 76]
 [25 46 65 87 96 18 53 78 14 35]
 [39 92 42 59 96 96 41 80 97 92]
 [54 62 72 43 62 38 94 63 31 58]
 [31 52 82 16 83 96 94 95 83 49]
 [13 21 10 75 65 20 45  5 97  0]
 [74 39 86 55 88 17 61 76 49 97]]
```

Task 2. (one standard deviation of the mean)

- Draw $n = 1000$ i.i.d. samples form $N(\mu, \sigma^2)$ with $\mu = 1$ and $\sigma^2 = 1$.
- Compute the empirical mean $\hat{\mu}$ and the sample standard deviation $\hat{\sigma}$ from the data. Compare $\hat{\mu}$ and $\hat{\sigma}$ with true parameters.
- Compute the proportion of samples that fall within one empirical standard deviation of the empirical mean, i.e., in the interval

$$[\hat{\mu} - \hat{\sigma}, \hat{\mu} + \hat{\sigma}].$$

Compare this proportion with the theoretical $\approx 68\%$. Can you compute this proportion in a single (vectorized) line?

Perform all computations without loops!

In [115...

```
n=1000
mi=1
sigma=1
arr=np.random.normal(mi,sigma,1000)

emp_mean=np.mean(arr)
```

```
emp_std=np.std(arr)
print(f"Empirical mean: {emp_mean}.\nEmpirical standard deviation: {emp_std}.")
print(f"Relative error - for mean: {round(abs(mi-emp_mean)/mi*100,2)}%, for standard deviation: {round(abs(sigma-emp_std)/sigma*100,2)}%.")
#we can see that those are relatively close to the true values of mi and std

print(f"Proportion: {np.sum(np.where(abs(arr-emp_mean)<=emp_std, 1, 0))/n}.")
print(f"Relative error to the theoretical 68% value: {round(abs(np.sum(np.where(abs(arr-emp_mean)<=emp_std, 1, 0))/n-0.68)/0.68,5)}%.")
```

Empirical mean: 1.017247375073365.

Empirical standard deviation: 0.9933898552520343.

Relative error - for mean: 1.72%, for standard deviation: 0.66%.

Proportion: 0.684.

Relative error to the theoretical 68% value: 0.00588%.

Task 3. (conditional operations on a random array)

For a 10×10 matrix of random floats in $[0, 1]$, perform this operation:

- If element $< 0.25 \rightarrow$ multiply by 0
- If element between 0.25 and 0.75 $\rightarrow$ square it
- If element $> 0.75 \rightarrow$ replace with 1

Print the original and transformed matrix.

Perform all computations without loops!

In [155...

```
b_matrix=np.random.random((10,10))
print(f"Original matrix:\n{b_matrix}")
b_matrix=np.where(b_matrix<0.25, 0, (np.where(b_matrix<=0.75,b_matrix**2, 1)))
print(f"\nTransformed matrix:\n{b_matrix}")
```

Original matrix:

```
[[0.21539153 0.93837149 0.25041471 0.9367662 0.84754086 0.41640858
 0.43367546 0.21906 0.540583 0.8325992 ]
[0.06374556 0.0454948 0.86292083 0.38814575 0.84832479 0.89678254
 0.87479048 0.53775491 0.80359336 0.34053589]
[0.89345002 0.86517719 0.41407625 0.04941909 0.79221559 0.76996206
 0.92333943 0.17230393 0.85340667 0.94627187]
[0.17511914 0.27747575 0.77250723 0.39179046 0.70058387 0.21046588
 0.20874693 0.00491456 0.88560183 0.24572189]
[0.4382003 0.47538964 0.58248812 0.59451314 0.912003 0.67707104
 0.80979581 0.58166881 0.25438373 0.65894705]
[0.4101169 0.76386935 0.53440622 0.83711346 0.97927239 0.36338581
 0.64707627 0.74391044 0.54633327 0.76527241]
[0.17807204 0.28414066 0.33596531 0.58609938 0.84896625 0.95379616
 0.44382223 0.56393409 0.37010114 0.62065001]
[0.05238642 0.49838154 0.7686702 0.03331028 0.62390415 0.20335717
 0.86129519 0.94942762 0.44080089 0.43983943]
[0.33422527 0.14415177 0.26008726 0.60094453 0.78574502 0.63254949
 0.14729488 0.6941001 0.95452219 0.49233951]
[0.45443172 0.59695086 0.6861114 0.47023031 0.24229125 0.43945078
 0.31614711 0.62395589 0.12662524 0.58352903]]
```

Transformed matrix:

```
[[0. 1. 0.06270753 1. 1. 0.1733961
 0.1880744 0. 0.29222998 1. ]
[0. 0. 1. 0.15065712 1. 1.
 1. 0.28918035 1. 0.11596469]
[1. 1. 0.17145914 0. 1. 1.
 1. 0. 1. 1. ]
[0. 0.07699279 1. 0.15349977 0.49081776 0.
 0. 0. 1. 0. ]
[0.1920195 0.22599531 0.3392924 0.35344587 1. 0.4584252
 1. 0.3383386 0.06471108 0.43421121]
[0.16819588 1. 0.28559 1. 1. 0.13204925
 0.4187077 0.55340274 0.29848005 1. ]
[0. 0.08073592 0.11287269 0.34351249 1. 1.
 0.19697817 0.31802166 0.13697485 0.38520644]
[0. 0.24838416 1. 0. 0.38925639 0.
 1. 1. 0.19430542 0.19345873]
[0.11170653 0. 0.06764538 0.36113432 1. 0.40011885
 0. 0.48177495 1. 0.24239819]
[0.20650819 0.35635033 0.47074885 0.22111655 0. 0.19311699
 0.099949 0.38932095 0. 0.34050613]]
```

Task 4. (Monte Carlo)

Let $f : [a, b] \mapsto \mathbb{R}$ be a measurable function with $-\infty < a < b < +\infty$. We aim to approximate

$$I(a, b) = \int_a^b f(x) dx$$

using

$$Q_n(a, b) = \frac{b-a}{n} \sum_{j=1}^n f(X_j).$$

where $X_1, \dots, X_n$ are i.i.d. random variables from $U(a, b)$.

Define a function `func(x)` that returns $f(x) = e^{-x^2}$ for a given input. The function must accept both a scalar and an array-like `x` and return the corresponding value(s).

Implement `mc_integral(a,b,n)` that returns $Q_n(a, b)$.

Evaluate it for $a = 0$, $b = 1$ and $n = 1000$ and compare with the true value $I(a, b) \approx 0.746824132812$.

Perform all computations without loops!

```
In [6]: def func(x):
        return np.exp(-x**2)

def mc_integral(a,b,n):
    sample=np.random.uniform(a,b,n)
    vect=np.array(func(sample))
    return (b-a)/n*np.sum(vect)

print(f"Our function works for both scalars (func(1)={func(1)}) and for arrays:\nfunc(np.array([1,3,0]))={func(np.array([1,3,0]))}.")
print(f"\nQ_n(a,b)={mc_integral(0,1,1000)}")

I_ab=0.746824132812
print(f"Relative error: {round(abs(I_ab-mc_integral(0,1,1000))/I_ab*100, 5)}%")
```

Our function works for both scalars (func(1)=0.3678794411714424) and for arrays:
func(np.array([1,3,0]))=[3.67879441e-01 1.23409804e-04 1.00000000e+00].

Q_n(a,b)=0.7409370195955264
Relative error: 0.99318%

Taks 5. (nearest points)

- Draw $n = 1000$ i.i.d. samples $X_1, \dots, X_n$ form from the bivariate normal distribution $N(\mu, \Sigma)$ with

$$\mu = \begin{bmatrix} 3 \\ 2 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 42, \\ 2, 3 \end{bmatrix}$$

Store the samples as rows in an array `X`.

- For each sample X_j , compute the Euclidean distance d_j to the mean μ .
- Find the 20 samples with the smallest distances d_j and store them (as rows) in an array `nearests` of shape `(20,2)`.
- Visualize the full dataset, the 20 nearest points, and the mean using:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 8))
plt.scatter(X[:, 0], X[:, 1], label='data', s=5, color='black')
plt.scatter(nearests[:, 0], nearests[:, 1], label='nearest', s=10, color='green')
plt.scatter(3, 2, label='mean', s=10, color='red')
plt.legend()
plt.show()
```

Perform all computations without loops!

```
In [81]: n=1000
mi=[3,2]
sigma=[[4,2],[2,3]]    #i'm gonna assume that "42," in sigma meant "4,2", so that sigma would be 2x2 matrix

X= np.random.multivariate_normal(mi, sigma, n)

new_arr=np.sqrt(np.sum((X - mi)**2, axis=1))
nearest_indices=np.argsort(new_arr)[:20]
nearests=X[nearest_indices]
print(f"Points nearest mean:\n{nearests}")
print(f"\nDistances for those points:\n{new_arr[nearest_indices]}\n")

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 8))
plt.scatter(X[:, 0], X[:, 1], label='data', s=5, color='black')
plt.scatter(nearests[:, 0], nearests[:, 1], label='nearest', s=10, color='green')
plt.scatter(3, 2, label='mean', s=10, color='red')
```

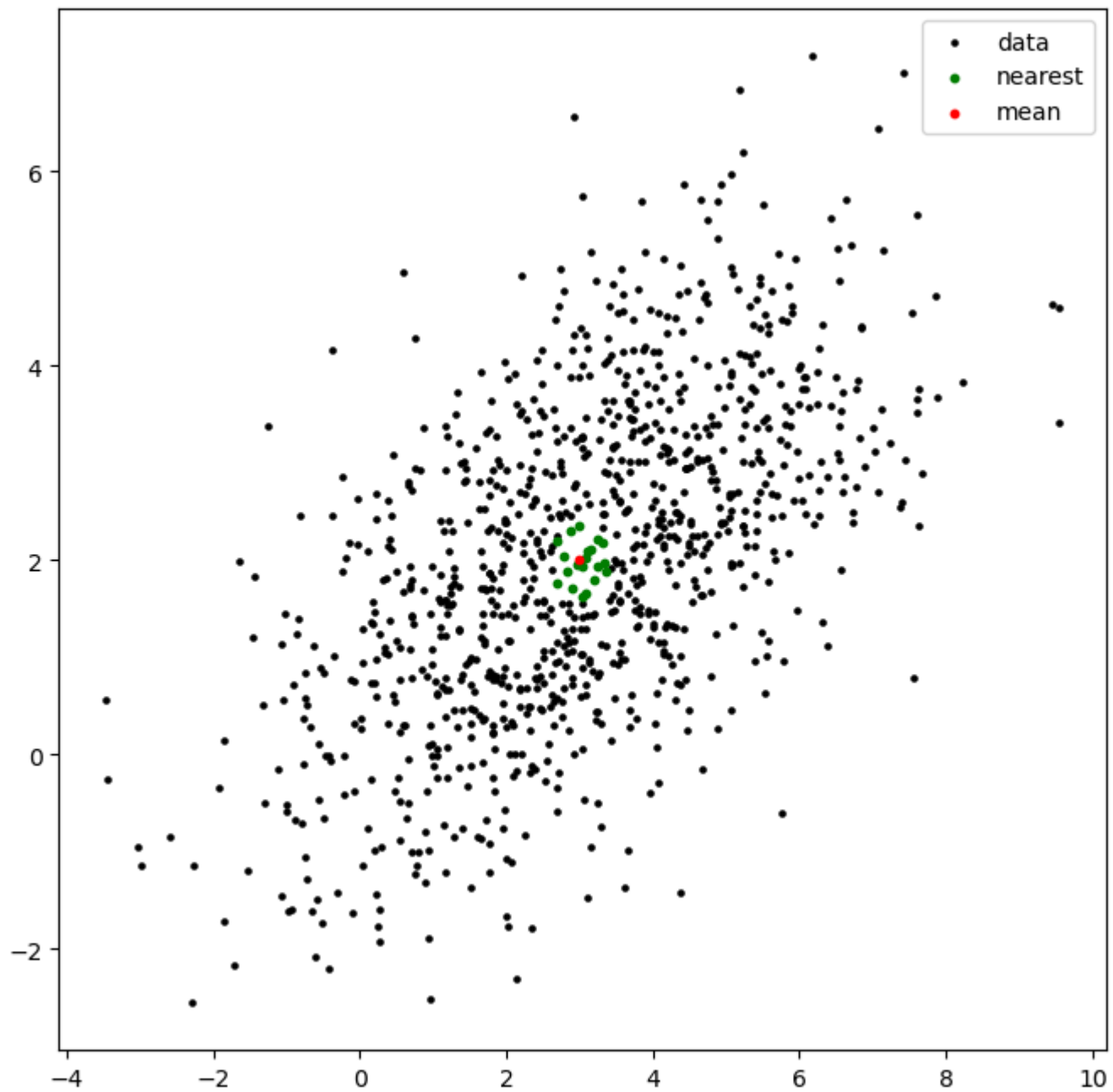
```
plt.legend()  
plt.show()  
  
print("\nAs we can see, we found the right 20 points.")
```

Points nearest mean:

```
[[2.97389592 1.95995015]  
 [3.04673823 1.93361242]  
 [3.08847535 2.03155096]  
 [3.10227309 2.08903138]  
 [3.15194983 2.10655241]  
 [2.82412187 1.88460806]  
 [2.78025776 2.04406927]  
 [3.24171569 1.92913417]  
 [3.20485178 1.79635162]  
 [2.89300861 1.71256084]  
 [2.88037234 2.29376027]  
 [3.24011877 2.22345885]  
 [3.34758784 1.97745442]  
 [3.3070946  2.17430087]  
 [3.07227567 1.65424163]  
 [2.98977097 2.35803052]  
 [3.34992609 1.89243236]  
 [3.03808958 1.63246872]  
 [2.68701697 2.20490086]  
 [2.70047408 1.76786807]]
```

Distances for those points:

```
[0.047806  0.08118974 0.09393269 0.13559635 0.18558601 0.21035307  
 0.22411772 0.25188974 0.28885448 0.30670577 0.31718429 0.32801048  
 0.34831826 0.35311172 0.35323168 0.35817662 0.36608614 0.36949974  
 0.37408921 0.37894723]
```



As we can see, we found the right 20 points.

Challenge (optimizing production)

This task is not mandatory!

A company manufactures three products (A, B, C) using four machines (M1, M2, M3, M4).

- The production time per unit (in hours) for each product on each machine is known.
- Each machine has limited available hours per day.
- Profit per unit of each product is also known.

to find the production plan (how many units of A, B, C to make) that maximizes total profit without exceeding machine time limits.

To do it do the following steps:

1. Generate data:
 - a. Randomly create a 4×3 matrix T where $T_{i,j}$ = hours machine i needs to make one unit of product j. Values between 0.5 and 4.0.
 - b. Create a 1D array capacity (size 4) with available hours per machine, random between 8 and 20.
 - c. Create a 1D array profit (size 3) with per-unit profits, random between 50 and 200.
2. Print T , capacity, and profit. Check their shapes and types.
3. Check how many hours each machine needs to produce one unit of each product, compare this with its available capacity, and determine how many machines are capable of producing at least one unit of each product per day.
4. Find the optimal production plan. Construct a grid of all possible integer production levels for products A, B, and C (from 1 to 5). From this grid, select only the plans that satisfy the machine capacity constraints. For each valid plan, compute the total profit and identify the one that maximizes profit. Display this optimal plan and its profit.

In []: